

Forecasting Bike Rental Demand Using Tree-Based Models

Course: UE24CS352A — Machine Learning

Team Members: Ibrahim Khaleel, Dheeraj

SRN: PES2UG25CS809, PES2UG25CS806

Date: October 2026

1. Problem Statement

Bike-sharing systems such as Washington D.C.'s Capital Bikeshare require accurate demand forecasts to manage bike availability, storage, and relocation costs. Our goal is to predict the **hourly count of bike rentals** using historical rental data, weather conditions, and calendar information. The evaluation metric is **Root Mean Squared Logarithmic Error (RMSLE)**:

$$\text{RMSLE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\log(p_i + 1) - \log(a_i + 1))^2}$$

where p_i is the predicted count, a_i is the actual count, and n is the number of observations. Lower RMSLE indicates better predictions. RMSLE penalizes under-predictions more heavily than over-predictions, which is appropriate for demand forecasting where stockouts are costlier than surplus. The log transformation also handles the wide range of rental counts (1 to ~977 rentals/hour) without over-weighting high-count hours.

2. Dataset Details

We used the **Kaggle Bike Sharing Demand** dataset, derived from the Capital Bikeshare program in Washington D.C. spanning 2011–2012.

- **Training set:** 10,886 hourly observations
- **Test set:** 6,493 hourly observations
- **Original features (12):** datetime, season, holiday, workingday, weather, temp, atemp (feels-like temperature), humidity, windspeed, casual, registered, count
- **Target variable:** count (sum of casual + registered rentals)

Feature type breakdown:

- *Categorical*: season, holiday, workingday, weather
 - *Continuous*: temp, atemp, humidity, windspeed
 - *Datetime string*: converted into month, dayofweek, hour
-

3. Approach

Our methodology followed three stages: feature engineering, model selection, and evaluation.

3.1 Feature Engineering

Based on insights from Du, He, and Zhechev (2014), we applied the following transformations:

1. **Datetime decomposition**: Split datetime into month, dayofweek, and hour, isolating the time-of-day signal.
2. **Peak-hour indicator**: Created a binary peak_hour feature:
 - Weekdays (Mon–Fri): 7–9 AM and 5–7 PM
 - Weekends (Sat–Sun): 10 AM – 6 PM
3. **Feature removal**: Dropped holiday (no predictive signal per the paper) and temp (highly correlated with atemp, causing collinearity).

3.2 Models Evaluated

We trained and 10-fold cross-validated **eight models** across four categories:

Category	Models
Linear	Linear Regression, GLMNet (Elastic Net), PCR
Kernel	SVR (RBF kernel)
Tree-based	Random Forest, GBM, CTree*
Ensemble	Stacking (RF + CTree → Linear meta)

CTree was approximated using scikit-learn's DecisionTreeRegressor, as the exact Conditional Inference Tree algorithm is available only in R's party package.

3.3 Additional Experiment

Following the reference paper, we also attempted **split regression** — training separate models on registered and casual counts and summing predictions — to test whether the two user groups respond differently to features.

4. Implementation Overview

The project was implemented in Python using pandas, numpy, and scikit-learn. The pipeline:

1. Load train.csv and test.csv
2. Apply feature engineering (decompose datetime, add peak-hour flag, drop unhelpful features)
3. Train each of the 8 models using 10-fold cross-validation
4. Compute mean CV RMSLE for each model
5. Select the best model, retrain on the full training set
6. Predict on the test set and save submission.csv
7. Visualize feature importances and model comparison

The RMSLE metric was implemented manually:

python

```
def rmsle(y_true, y_pred):  
    y_pred = np.maximum(y_pred, 0)  
    return np.sqrt(np.mean((np.log1p(y_pred) - np.log1p(y_true)) ** 2))
```

5. Results

Model	10-Fold CV RMSLE
Split Regression (reg + cas)	0.3928
Random Forest	0.3985
CTree (DecisionTree substitute)	0.4469
GBM (Gradient Boosting)	0.5016
Stacking Ensemble	0.5624

Model	10-Fold CV RMSLE
SVR (RBF kernel)	0.8452
GLMNet (Elastic Net)	1.1629
PCR	1.1817
Linear Regression	1.1855

Key findings:

- **Tree-based models dominated.** Split regression and Random Forest achieved the lowest CV RMSLE (0.3928 and 0.3985 respectively). The gap between them is within cross-validation noise, so they are effectively equivalent.
- **Time-of-day is the dominant signal.** Feature importance analysis showed `peak_hour` (0.42) and `hour` (0.22) account for **~64% of the model's decision-making**.
- **Weather plays a secondary role.** `atemp` (0.14) and `humidity` (0.07) contribute meaningfully but are less predictive than time-of-day.
- **Linear models failed.** Bike demand is highly non-linear (e.g., hour 8 AM $\neq$ 8 $\times$ hour 1 AM), which linear models cannot capture. All three linear variants (Linear, PCR, GLMNet) had RMSLE > 1.16.
- **Stacking underperformed.** Our base models (RF + CTree) were both tree-based, so they made similar errors. Stacking works best with diverse base models.
- **Split regression performed well.** It slightly edged out direct count regression, though the difference is negligible. This contrasts with the reference paper, where split regression performed noticeably worse.

6. Conclusions

Tree-based models, particularly Random Forest and split regression, significantly outperformed linear and kernel approaches for this problem. Our best models achieved a CV RMSLE of **0.3928**, beating the reference paper's benchmark of 0.503. The most predictive feature was `peak_hour`, followed by `hour` and `atemp` — confirming that **time-of-day patterns are the dominant driver of bike rental demand**, with weather conditions playing a secondary but non-trivial role.

Limitations and Future Work:

- The 10-fold CV uses shuffled data, which mixes past and future observations. A **time-series split** would better reflect real-world deployment.
 - **True CTree** (via R's party package) may yield slightly different results than our DecisionTree substitute.
 - **GLMNet with Poisson loss** (rather than Gaussian) could improve the regularized linear model, since our target is a count variable.
 - **Hyperparameter tuning** for GBM could close the gap with Random Forest.
-

7. References

1. Du, J., He, R., & Zhechev, Z. (2014). *Forecasting Bike Rental Demand*.
2. Friedman, J., Hastie, T., & Tibshirani, R. (2010). Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software*, 33(1).
3. Liaw, A., & Wiener, M. (2002). Classification and regression by randomForest. *R News*, 2(3).
4. Hothorn, T., & Zeileis, A. (2006). Unbiased recursive partitioning: A conditional inference framework. *Journal of Computational and Graphical Statistics*, 15(3).
5. Kaggle. *Bike Sharing Demand*. <https://www.kaggle.com/c/bike-sharing-demand>